Prompt Log — CourseMate

Course: BUS4012 – Vibe Coding for Startups, Assignment 3 **Tool used:** Claude Code (Anthropic) **GitHub repository:** https://github.com/Sneha1010-code/Coursemate **Live app:** https://coursemate-five.vercel.app **Live API:** https://coursemate-1-i4m1.onrender.com

This log records the prompts used to develop CourseMate, in the order they were given, each with the intent behind it and what it produced. Development followed an iterative, conversational workflow: an initial detailed brief set the scope and constraints, and each subsequent prompt either supplied new information (credentials, confirmation that a step was complete) or directed the next stage of work. Short confirmation prompts are included alongside the main ones for completeness and are annotated with the decision they were authorizing.

---

1. Project brief and build authorization

> *"You are my autonomous software development assistant... [full requirements: read the assignment PDF and reference images, treat the PDF as the source of truth, build the entire application end-to-end using React + Python + Supabase, deploy it, don't stop after a partial implementation, ask only when genuinely blocked on credentials/config, never commit secrets, use a proper .gitignore, confirm before first push]"*

Intent: Establish the full scope of the assignment in a single brief — read the spec and reference screenshots, define the technology constraints, and set the expectation that the agent should deliver a complete, deployable application rather than a partial prototype, flagging only genuine blockers.

Outcome: The assignment PDF (`assignment_req/vibecodingassignment.pdf`) and reference mockups (`Reference_images/`) were read and used to identify the required stack (React + FastAPI + Supabase) and the five-screen signup → course details → profile setup → browse matches → pending requests flow. This produced the initial architecture decision: Supabase Auth handles login directly from the frontend using the public anon key, while all data access is routed through a FastAPI backend using the private service-role key, so no sensitive credential is ever exposed to the browser.

2. Plan confirmation and implementation go-ahead

> *"ok start"*

Intent: After reviewing the proposed architecture (React + FastAPI + Supabase, Vercel + Render deployment) presented back by the agent, authorize implementation to begin without further upfront discussion.

Outcome: Node.js was installed locally via nvm, the Vite React app and FastAPI backend were scaffolded, the Supabase schema (`profiles` , `connections` tables with row-level security enabled and no client-facing policies) was written, and all six screens (Sign Up, Log In, Course Details, Profile Setup, Browse Matches, Pending Requests) were built to match the reference mockups' layout, typography, and colour scheme.

3. Supabase credentials provided

> *"SUPABASE PROJECT URL: ... SUPABASE PUBLISHABLE KEY: ... SUPABASE SERVICE_ROLE KEY: ..."*

Intent: Supply the real Supabase project credentials so the application could be connected to and tested against a live database rather than placeholder values.

Outcome: Local `.env` files for both frontend and backend were populated with the credentials (never committed — both files are listed in `.gitignore`), followed by a request to run `backend/schema.sql` in the Supabase SQL Editor to create the `profiles` and `connections` tables.

4. Confirmation to proceed with end-to-end testing

> *"done" / "done, run it now"*

Intent: Confirm the database schema had been applied so that automated end-to-end testing against the live database could proceed.

Outcome: A full two-user, Playwright-driven test was run through the real UI (sign up → course details → profile setup → browse matches → connect → accept), which surfaced two real issues that were diagnosed and fixed in the same pass:

1. Supabase's default "Confirm email" setting blocked sign-ups behind an email-confirmation step.
2. After disabling that, sign-ups still failed with `over_email_send_rate_limit` — the default shared Supabase SMTP service has a very low hourly email quota, and the public `signUp()` endpoint is rate-limited against it regardless of the confirm-email setting. Left unfixed, this would have blocked sign-ups live during the demo recording as well, not just during testing.

   Fix: Added a `POST /auth/signup` backend endpoint (`backend/app/routers/auth.py`) that creates the user via Supabase's admin API using the service-role key (`email_confirm: true`, no email sent, not subject to the rate limit), inserts the profile row, and lets the frontend immediately sign the user in with `signInWithPassword()`. This also removes a flaky third-party dependency from the signup path, making it more reliable for the demo recording.

The full flow was then re-verified end-to-end with two accounts, and the resulting `profiles` and `connections` (`status: accepted`) rows were confirmed directly via the Supabase API to ensure the data was genuinely persisted, not just reflected in the UI. Test accounts were deleted afterward to leave the project clean for the demo recording.

---

5. GitHub publication — structured history, public repository

> *"yea the repo is public, push it with necessary humanized commits"*

Intent: Publish the project to GitHub as a readable, milestone-based commit history (one meaningful commit per development stage) instead of a single bulk commit, and confirm the repository should be public for the assignment's GitHub walkthrough requirement.

Outcome: The initial single commit was restructured into 9 commits grouped by concern (scaffolding, backend core, backend endpoints, schema, the signup fix, frontend design system, auth screens, matching screens, documentation), then pushed. This required first resolving a GitHub authentication mismatch — the machine's existing SSH key belonged to a different account than the repository owner — by installing the `gh` CLI and authenticating as the project owner (`gh auth login`) before the push could proceed with write access.

6. Backend deployment to Render

Intent: Deploy the Python backend to a cloud platform to satisfy the assignment's deployment requirement.

Outcome: A Render Web Service was connected to the repository (`backend/render.yaml`). The initial build failed because Render's default Python version (3.14) has no prebuilt wheel for `pydantic-core` , and compiling it from source failed since the underlying Rust bindings (PyO3) don't yet support 3.14. This was resolved by pinning `PYTHON_VERSION=3.12.8` via `backend/.python-version` and a matching Render environment variable — the same version used throughout local development and testing.

7. Frontend deployment to Vercel and CORS resolution

Intent: Deploy the React frontend to Vercel and connect it to the live backend.

Outcome: A Vercel project (root directory `frontend`) was configured with the three required `VITE_*` environment variables. Signing up on the deployed app initially failed with a network error, traced to the backend's CORS allow-list (`FRONTEND_URL`) still pointing at `localhost` — resolved by updating it to the Vercel production URL. A second CORS error was then traced to testing from a Vercel *preview* deployment URL (a random per-commit subdomain) rather than the stable production URL the backend was configured to allow; pointing testing at the production URL (`https://coursemate-five.vercel.app`) resolved it. The fix was verified end-to-end by confirming a submitted profile landed correctly in the Supabase `profiles` table on the live, deployed stack.